Zurich University of Applied Sciences
Department Life Science and Facility Management
Institute for Computational Life Sciences

Sensitivity Analysis of a TIR-Based Cold-Water Patch Detection Algorithm

The Influence of Cold Water Tributaries on Cold Water Patches

Bachelor Thesis

by
Dürig Etienne

Bachelor program Applied digital Life Sciences
Focus Digital Environment
Study cohort 2023

June 13, 2026

Dr. Manuel Antonetti
ZHAW Life Sciences und Facility Management
Institute for the Environment and Natural Resources
Research Group: Ecohydrology
Campus Grüenthal, 8820 Wädenswil

Dr. Norman Juchler
ZHAW Life Sciences und Facility Management
Institute for Computational Life Sciences
Research Group: Medical Image Analysis and Data Modeling
Schloss, 8820 Wädenswil

0.1 Theoretical Background

0.1.1 Model - Working Definition

The term model is widely used across environmental and earth sciences, yet a precise definition of a model is surprisingly challenging to find in literature. In this work, a model is understood as a set of simplified laws and processes, intertwined by dependencies, that together form a reduced reflection of an underlying system. The aim is to capture the essential behavior of that system and enable analysis from which inferences about reality can be drawn. This view is consistent with Saltelli et al. (2008), who, drawing on Rosen (1991), describe the modelled portion of reality as an arbitrary enclosure of an otherwise open and interconnected system.

0.1.2 Model Input Sensitivity Analysis

The inputs to models can diver and are generally dependent in their structure and many-foldness on the nature of the model itself. Under the light of sensitivity analysis however, it can be worthwhile to define, what the inputs to a model actually contains. In sensitivity analysis, everything that causes variation in the output of the model is classified as an input (Satelli_Marco). Providing an example. If a linear regression of a target variable Y is performed, then not only are the predictors $X_1, \ldots, X_n$ considered as model inputs, but also the parameters $\theta_1, \ldots, \theta_n$ that correspond to the predictors.

0.1.3 Sensitivity Analysis

According to Saltelli et al. sensitivity analysis is "The study of how uncertainty in the output of a model [...] can be apportioned to different sources of uncertainty in the model input" (Saltelli et al., 2004). Sensitivity analysis provides a range of benefits. Firstly it allows the Modeler to apply a strategy called "Factor Fixing". If we discover, that the model output Y is independent of one or multiple parameters θ_i, then we can set these to fixed constants. It may then even be possible to condense a complete submodule of a model, if all parameters connected to this sub-module are noninfluential. So sensitivity analysis can be a tool for model simplification (Saltelli_Marco). Secondly, sensitivity analysis generally improves model validity by identifying highly influential and therefore critical model components (Ligmann-Zielinska). A range of different methods for sensitivity analysis exist. They can be grouped into global and local analysis methods.

Local methods compute a sensitivity score for each parameter at a single point in the input space, typically the baseline point, which is customarily the best estimate point for the parameters. Scores are based on partial derivatives and are therefore only informative for linear models, where the derivative at one point represents behaviour across the entire input space (Saltelli and Annoni 2010).

Global sensitivity analysis methods explore the entire input space by varying all input parameters simultaneously across their plausible ranges. Sensitivity scores are therefore not tied to a single point but reflect average behaviour across the full parameter space. This makes global methods valid regardless of model linearity, as no assumption about model structure is required. (Saltelli and Annoni 2010).

global methods

The Morris Method

The Morris Method belongs to the group of screening methods in sensitivity analysis. These are ideal to filter a short list of important input parameters from computationally expensive models, with a great number of input parameters. This can be done, as these methods only require a relatively little number of model evaluations to provide sensitivity measures for the input parameters. This makes these methods a good choice when the target of the sensitivity analysis is Factor Fixing. (Satelly and Tarantola). The following explenation of morris method closely follows Saltelli_Marco and is explained more explicitly in their chapter.

At the core of this introduction sits a k-dimensional model Y with inputs $X_1, \ldots, X_k$. These inputs all together can also be written in a vector $\mathbf{X}$. The model inputs are defined in the k-dimensional unit cube, meaning that X_i can take on a value in the interval $[0, 1]$ (Saltelli, Tarantola). It is worth pointing out, that every model with a continuous input can be scaled to the $[0, 1]$ range, which makes the unit cube a great place to define methods (Claude). In the Morris method, this input space is discretized into a grid Ω. This grid features p levels.

An elementary effect of the ith input variable is then defined as:

$$EE_i = \frac{Y(X_1, X_2, \ldots, X_{i-1}, X_i + \Delta, \ldots, X_k) - Y(X_1, X_2, \ldots, X_k)}{\Delta} \tag{1-1}$$

Where Δ represents some step in the unit cube. Δ can be chosen from a range of different values, which are given by the set $\left\{\frac{1}{p-1}, \ldots, 1 - \frac{1}{p-1}\right\}$. Which value to choose for Δ from this set has implications and will be addressed in a later paragraph.

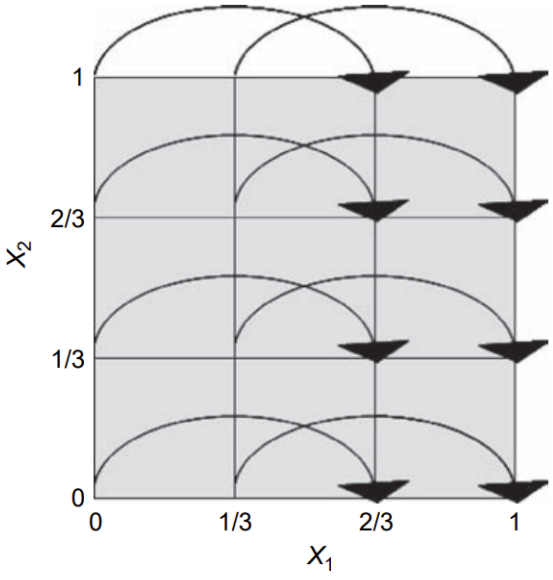

Abb. 1-1: An example grid Ω. The grid features two parameters X_1, X_2, meaning that it is a two-dimensional grid. Further, it features four levels ($p = 4$). The step length Δ chosen here is $\frac{2}{3}$, which corresponds to an element of the set of viable values for Δ: $1 - \frac{1}{4-1} = 1 - \frac{1}{3} = \frac{2}{3}$.

In the definition of the elementary effect, the vector $\mathbf{X} = (X_1, X_2, \ldots, X_k)$ has to fulfil the condition that $\mathbf{X} + \mathbf{e}_i \cdot \Delta$ still has to lie within the grid Ω. $\mathbf{e}_i$ is a vector of zeros except

having a one at its ith entry. So for instance:

$$\mathbf{e}_2 = (0, 1, 0, \ldots, 0) \tag{1-2}$$

The Morris method occupies itself with finding the statistical distribution of each of the i elementary effects. $\mathcal{F}_i$ is thereby used to denote the distribution of EE_i, in other words $\mathcal{F}_i \sim EE_i$. $\mathcal{F}_i$ is a finite discrete distribution, and its number of elements depends on the number of grid levels p, which is however not that much of interest in an applied setting.

How are the EE_i sampled? Generally many different points $\mathbf{X}$ over the whole grid are sampled and then used to compute elementary effects EE_i. This guarantees, that each elementary effect EE_i is computed for many areas within the grid. This is also the reason, why the Morris method is generally classified as a global method. The exact sampling scheme is out of this project's scope but is covered in detail in Saltelli_Marco chapter 3.3.

Once the distributions $\mathcal{F}_i$ have been mapped out, statistical measures can be computed just like over every other statistical distribution. In Morris, the mean μ_i and the standard deviation σ_i of $\mathcal{F}_i$ are computed. These two statistical measures together already allow for reasonable insight into the influence of the ith parameter X_i which corresponds to the ith elementary effect EE_i:

If μ_i deviates strongly from 0, then this means that the elementary effect EE_i generally takes on non-zero values and therefore that the model output changes notably when a step of Δ is performed along the dimension spanned by X_i. This then ultimately means, that changes in X_i seem to generally affect the model output Y considerably and X_i therefore receives a high sensitivity score. Generally, the more μ_i deviates from zero, the larger the sensitivity of the model output towards X_i.

If σ_i is large, this means that the distribution $\mathcal{F}_i$ is spread out considerably over a larger range. This indicates, that the elementary effects EE_i that were computed were generally different depending on their location in the unit cube. Meaning that in one area, the model might only yield small changes in output, when X_i is varied while in other areas X_i leads to strong changes in model output. This behaviour points either to nonlinearity regarding X_i or to the fact that X_i interacts with other input parameters. Which effect is responsible for the high standard deviation (or whether both are responsible for the effect at the same time) can not be determined. Which is a shortcoming of the method of Morris. Figure 1-2 provides an visual example of possible F distributions of a Factor X_i with the associated μ and σ values and the corresponding interpretation.

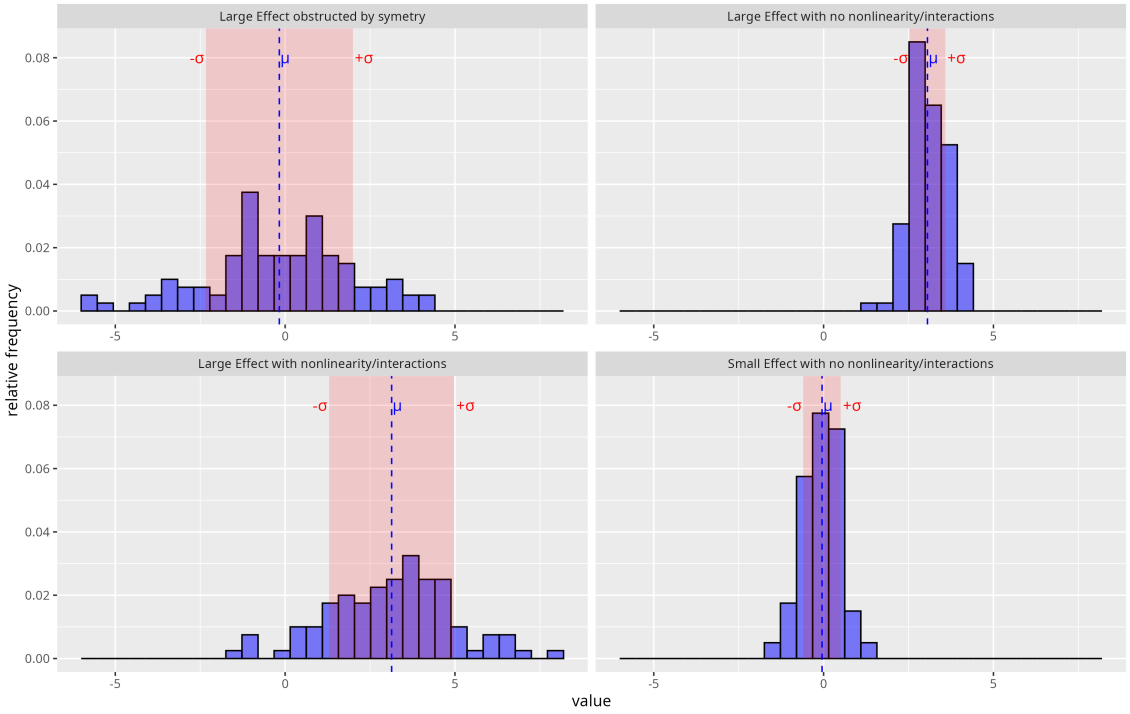

Abb. 1-2: Different F distributions of a parameter X_i are shown. Additionally, μ and σ of each distribution is provided. An interpetation is provided

Situations can occur, where $\mathcal{F}_i$ contains many extreme values however the distribution is symmetric around zero. Then computing the mean value μ_i will yield a value close to zero and we would assume, that the parameter X_i therefore is noninfluential. This would be a faulty assumption, as the sampled elementary effects EE_i are substantial in their magnitude but just cancel themselves out. This phenomenon can be spotted by always taking σ_i into consideration, as in this scenario σ_i is large, but it is more common to also sample an additional distribution $\mathcal{G}_i$ which captures the magnitude of the elementary effects $|EE_i|$. So $\mathcal{G}_i \sim |EE_i|$. We then compute the mean of this distribution μ_i^* which can be interpreted like μ_i. The further away from zero, the more influential is the parameter. But since it is the magnitude, the distribution never features negative values and therefore cancellation effects can not occur.

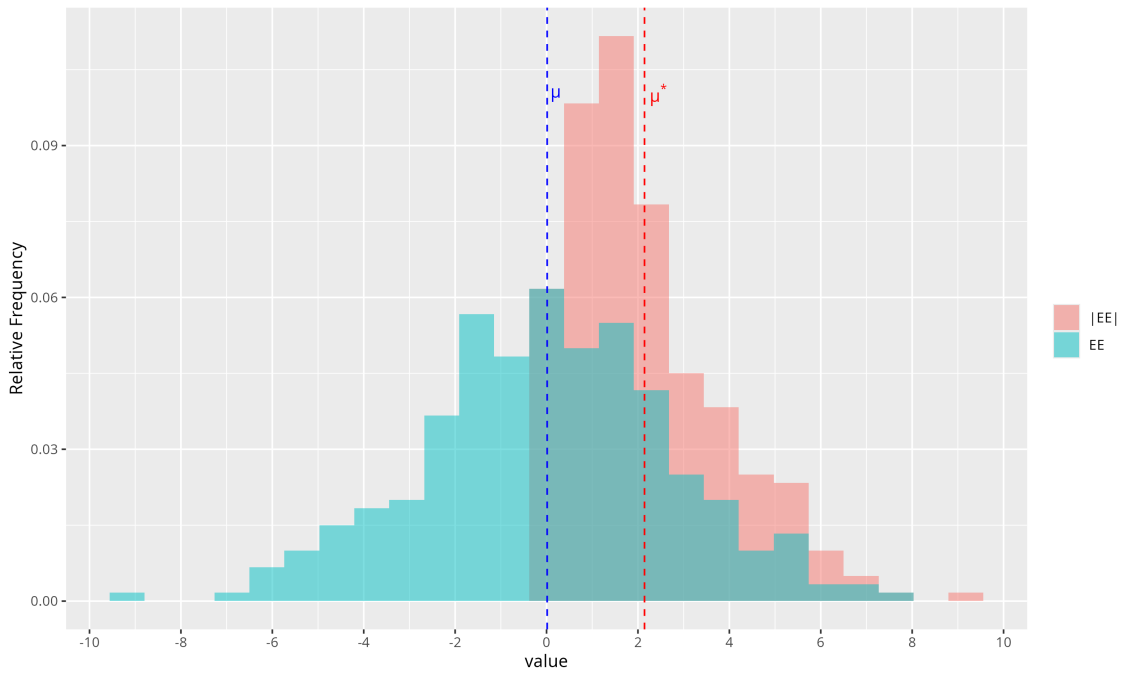

Abb. 1-3: The difference of μ and c visualized with their underlying distribution $\mathcal{G}$ and $\mathcal{F}$. While the mean μ of the distributuion $\mathcal{F}$ lies at around zero, $\mu^\star$ significantly deviates from zero

0.1.4 Coldwater Patches

Cold water patches are areas in rivers and streams that feature a colder water temperature than the temperature of the "hauptgerinne". The temperature difference varies in literature but is typically set to be between 0.5 and 3 degrees Celsius below the "hauptgerinne" temperatur, but this varies over the literature (z.B. Dugdale et al., 2013, 2015; Wawrzyniak et al., 2016; Casas-Mulet et al., 2020). Cold water patches are not automatically a cold water refugia. These are coldwater patches which are also used by coldwater organisms for thermal relief (Meja et all.). The importance of cold water refugia is expected to rise with the effects of climate change (IPCC, 2014)

0.2 Data

Chapter 1

Methods

The presented methods can be divided into 3 logical steps. The first step focuses on optimizing the currently available CWP detection algorithm with respect to execution speed, to set the stage for for the sensitivity analysis. The second step focuses on deriving sensitivity indices using the morris method for a set of variable CWPs which can be clearly attributed to beeing tributary-caused or non tributary-caused. The third step of the method then focuses again on adjusting the CWP algorithm, this time with respect to the insight generated from observing the sensitivity to different parameters. This three step division of the methods is also provided as a graphical overview in Fig 0-1.

Abb. 0-1: High-level overview of the methodological workflow.

The first optimisation step and the seccond step of the methods was performed based on the data collected of the emme unterlauf. The sensitivity analysis is performed using all the data from both segments of the emme unterlauf aswell the segment of the emme oberlauf.

1.1 Optimizationn of Existing Algorithm

most sensitivity analysis methods feature a high cost when it comes to model executions. Morris, is one of the cheaper methods but still reqires a reasonable amount of model executions. A model with 3 parameters and r=50 reqires 50 *(3 + 1) = 200 executions. To perform these executions in a feasonable timespan for the provided CWP algorithm optimisation of the computations is needed. Additionally parallel execution of the algorithm should be performed to further reduce execution times for the sampling. For the latter the zhaws High perfomance cluster is beeing used which has direct implications on

the optimisation approach. To enshure that the code optimisation speed gains also carry over to the infrastructure (e.g. the underlying distributed file system) of the hpc, the optimisation and benchmarking is performed directly on the hpc system.

1.1.1 Reimplementation of the CWP algorithm based on GDAL and system calls

The current algorithm focuses on per slab processing of the raster using a set of R loops and relying on terra and sf functions to implement the logic. The reworked algorithm is disk/file based and deployes GDAL over its command line interface directly. This decission was choosen, as terra operations are are somethimes not to performant and GDAL offers a often faster set of geocomputational tools which can be called from R using system() or sf::gdal_utils() (Lovelace et al., 2019). Further, empirical testing in a prior coursework exercise also showed a significant speedup of GDAL-based raster operations compared to `terra` across a range of use cases (Dürig, 2026). This coursework was not peer-reviewed and is cited here only as a supporting empirical observation. Additionally the GDAL approach was also choosen, as it enables easy chunk processing with reading and writing to disk, which helps to keep memory foodprint small. This can be beneficiial in a HPC setting, where not only the number of available cores but also the available memory can be comme an issue in performing computations. The draw back of this decission is that writing intermediate results to disk causes large spikes in disk space consumption, as intermediate reasters of the algorithm can be up to 25 GB large. This was counteracted by deploying compression during writing and decompression during reading, a strategy that only produced little computational overhead but greatly reduces the size of the intermediate results. Speciffically, this can be implemented by setting the compression options `COMPRESS=DEFLATE` and `PREDICTOR=2` when working with GDALs geotiff driver (GDAL/OGR contributors, 2026). Lastly it should also be pointed out that heavily relying on gdal introduces a lot of I/O activity, which can significantly hurt performance on distributed file systems. To further optimize execution speed, `terra::extract()` was replaced with `exact_extract()` from the `exactextractr` package. This was done as `exact_extract()` extracts aggregated values from rasters considerably faster than `terra::extract()` for large rasters (Mieno, 2022); it should be noted, however, that this speed difference was not tested in isolation within this work. A further difference between the two functions concerns how partially covered cells are handled: `exact_extract()` performs a coverage-fraction-weighted extraction, in which partially covered cells are weighted by their fraction of coverage (ISciences, LLC, 2026), whereas `terra::extract()` by default includes a cell only if its center point falls within the polygon (Hijmans et al., 2024) – the behaviour used in the original (v1) implementation. To approximate this cell-center-inclusion behaviour within `exact_extract()`, only cells with a coverage fraction of at least 50% were included. This approximation is not exactly equivalent to terra's center-point rule.

Table 1-1 provided a high level overwiev over the reimplemented cwp detection algorithm. The Description columns lists the computational steps of the algorithm, while the Tool / Function lists the main imortant function used in this step.

Tab. 1-1: High-level overview of the reworked CWP detection algorithm, including the primary tool or function used for each step.

Step	Description	Tool / Function
1	Divide the river reach into n centerline segments; construct a wide zone polygon and a narrow reference-strip polygon for each segment.	`sf::st_buffer`
2	Rasterize the zone polygons to a raster matching the TIR raster resolution, using the segment id as burn-in value.	`gdal vector rasterize`
3	Round the TIR raster using a fixed rounding factor.	`gdal raster calc`
4	Extract the median temperature per segment from the rounded TIR raster within each reference strip; store in a look-up table keyed by segment id.	`exactextractr::exact_extract`
5	Reclassify the zone raster, replacing each segment id with its reference temperature from the look-up table, yielding the reference raster.	`gdal raster reclassify`
6	Subtract the rounded TIR raster from the reference raster; flag pixels where the difference exceeds ΔT, producing a binary CWP raster.	`gdal raster calc`
7	Polygonize the binary CWP raster into patch polygons.	`terra::as.polygons`
8	Filter patches by minimum area, compute per-patch temperature statistics, and apply a patch-level threshold check against the reference temperature.	`exactextractr::exact_extract`

1.1.2 Output eqivalence and Runtime Comparison

The resulting reimplementation was tested for equivalence in output by comparing the produced CWP polygons to the CWP polygons of the old algorithm when deployed with the exact same set of parameters. Table 1-2 shows the parameter combinations deployed in testing. Further, the runtime between the new algorithm and the existing version was compared with respect to the step length parameter. Runtime was compared across three segment lengths (400, 800, and 1200 m), with all other parameters held constant (zone halfwidth = 60 m, buffer = 3 px, $\Delta C = 1.0\,^{\circ}\mathrm{C}$, minimum patch area = 2 m^2, rounding = $0.1\,^{\circ}\mathrm{C}$, 8-connectivity enabled).

Tab. 1-2: Parameter combinations used for the output equivalence comparison between v1 and v2.

Parameter	Test 1	Test 2	Test 3
`segment_length_m`	200	800	1200
`zone_halfwidth_m`	60	60	60
`buffer_px`	4	3	3
`delta_C`	0.5	1.0	1.0
`min_patch_area_m2`	2	2	2
`round_to`	0.1	0.1	0.1
`connect_diagonals`	TRUE	TRUE	TRUE

1.2 Sensitivity Analysis using Morris

With the algorithm optimized, sensitivity analysis can now be carried out to determine the sensitivity of the CWP detection algorithm with respect to the parameters buffer pixel count, step length and temperature delta. Especially of interest is to observe the difference in sensitivity towards the step length parameter between tributary-caused and non tributary-caused CWP. For this reason a pre selection method is deployed to guarantee that the studied CWP are sensitive towards the step length parameter. Senistivity towards pixel buffer count and temperature delta are also of interest. (Tonolla et all.) allready found in their sensitivity analysis, that the buffer pixel count seems to have little influence on the CWP area. If this finding can be replicated algorithmic simplifcation using factor fixing (refference to theory) might be possible. Additionally, the temperature delta parameter is among the most influential according to Tonolla et. all. It is of generally interest a tool to determine the validity of the method, to check wheter the same is found. The general approach can be logically segmented into two steps, section 1 focues on determining CWP that are succeptible to the step length parameter and classifying these cwp into tributary-caused and non tributary-caused CWP. The second step focuses on deriving morris sensitivity indices for these CWP with respect to their area.

1.2.1 Determining and classifying step length sensitive CWP

Figure 2-2 shows a high level overview of the first step. First a morris design with r=44 and k=2 is step up for the buffer pixel count and step length parameters. Then the speed optimized CWP algorithm is deployed on the resulting parameter tuples. Lastly the results are aggregated to a fuzzy representation of the CWP over all runs, to then select CWP which are variable to the CWP algorithm and to classify them into tributary-caused and non-tributary caused CWP.

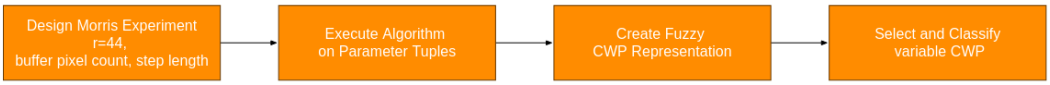

Abb. 2-2: TODO: describe what this overview plot shows.

Setting up first Morris experiment

First, a two dimensional morris sampling grid was set up spanning two parameters buffer pixel and step length. A total of 6 levels where set for each parameter. the step length varying from 400 m to 1200 m and the number of pixels in the buffer spanning from 2 to 7. To compute the partial differences a jump distance of 3 was set. Fig shows the deployed grid visualized in greater detail with the jump distance additionally drawn in.

From this grid a total of 44 trajecotries where sampled (r=44) which results in a total of 66 * (2 + 1) = 132 parameter touples beeing generated. Each tuple holds a different set of buffer pixel count and step length. These where written to a file along with a unique identifier of the tuple. For the sampling procedure the R package "sensitivity" was deployed. This package additionally produces an object representing the morris sampling grid and can later be provided with the results to carry out the computation of the morris indices automatically. this object was saved to an RDS and stored on disk for later computation.

Model Evaluation along Parameter Combinations

Next a parallelized computation scheme was set up to evaluate the CWP algorithm along the created parameter tuples while keeping the other parameters not part of the morris experiment constant. Table x shows to which values the other parameters where held constant.

To orchestrate the parallel execution of the CWP algorithm, the open sourche software GNU parallel was deployed (Tange, 2024). Specifically gnu prallel was used to implement a worker pool of processes that each execute the cwp algorithm with a given parameter tuple they fetch from a parameter text file. Before starting the computation, the processes also set up an individual working directory where intermediate results aswell as the final result are written to. To keep the parameter combination aswell as the id present and the results therefore identifiable, the folder name holds the id aswell as the parameters.

Figure 2-3 visualizes the GNU prallel workflow deployed as an execution graph in time where time is encoded in the y-dimension of the plot.

Creation of a fuzzy overlay of CWP

the resulting cwp polygon layers where then processed into an singular raster over a series of processing steps. Firs each polygon layer was rasterized using the same pixle size as the TIR rasters. Then these different raster representations of the cwp where staked into a multy layer raster and a pixel whise mean calculation was performed. This aggregates the threee dimensional raster stack into a two dimensional raster. The pixel whise mean computation produces a pixel score between 0 and 1 which can be interpreted the following way. A pixel value of 1 indicates that this area of the emme basement, was part of a cwp during each of the runs. A pixel value near zero indicates that the area covered by the pixel almost never was part of a cwp in the model outputs. The resultign raster can be understood as a fuzzy membership representation of the CWPs in the emme where each pixel has a probability score of belonging to a cwp. From these fuzzy cwps it can also be read which cwps are variable with respect to the step lengt (or the buffer pix => which hoverwer has a neglectable influence as seen in the results). CPWs which show a lot of

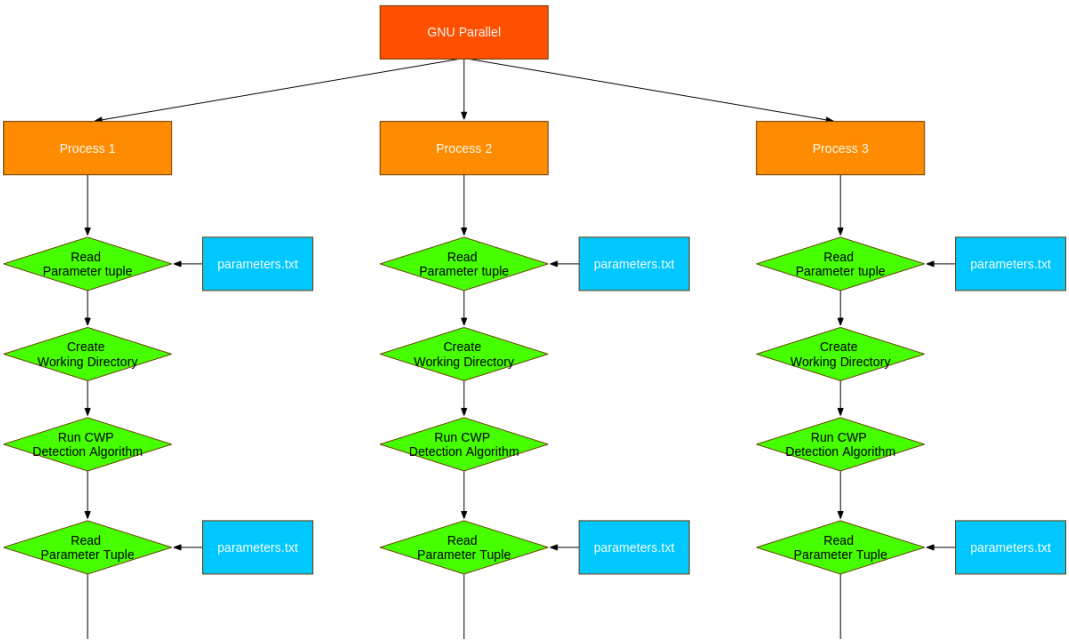

Abb. 2-3: TODO: describe what this plot shows.

differnt shades in their colourling, hold pixels which are generally variable in their identiy as cwp pixel or non cwp pixel. These are then cwp that seem to vary with respect to bla bla

Selection of CWP

1.2.2 Morris Experiment for pixel buffer, step length and temperature delta

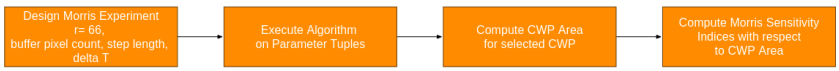

Abb. 2-4: TODO: describe what this overview plot shows.

Compute total area statistics for selected CWP

1.2.3 Comparison of outputs

1.2.4 Speed comparison in execution

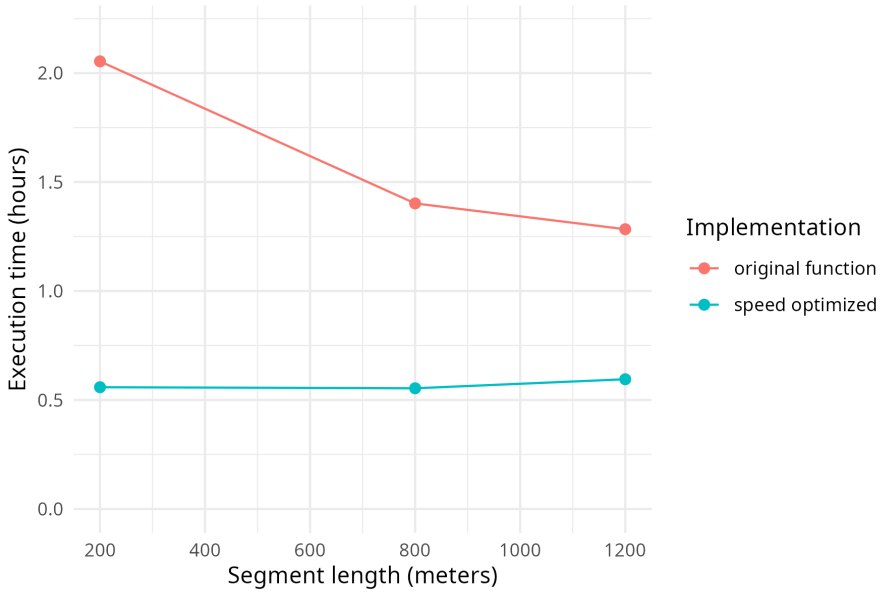

Abb. 2-5: Execution time of the original (v1) and optimized (v2) implementations of `detect_cwp_single` as a function of segment length. The optimized implementation shows execution times that are both shorter and largely constant across segment lengths, whereas the original implementation scales disproportionately with decreasing segment length (i.e., increasing number of segments).

1.2.5 Sensitiviy scores for step length parameter

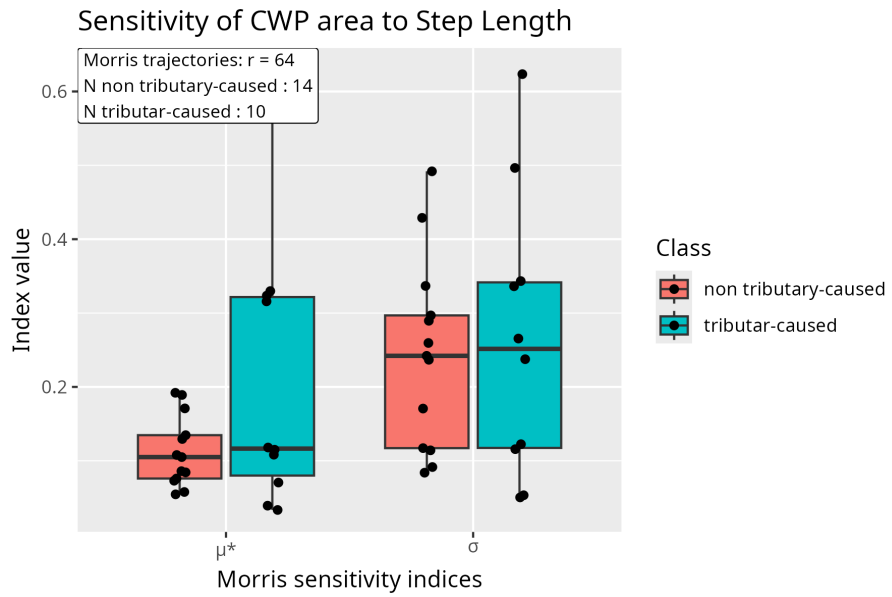

Abb. 2-6: Morris sensitivity indices (μ^* and σ) for the `step_length` parameter with respect to cold water patch area, stratified by patch class (red: tributary-caused; blue: non-tributary-caused). Boxplots show the distribution across bounding boxes, with sample sizes (n) given below each box.

1.2.6 Mu star for all parameters over all CWP

1.2.7 Mu.star and size for every cwp

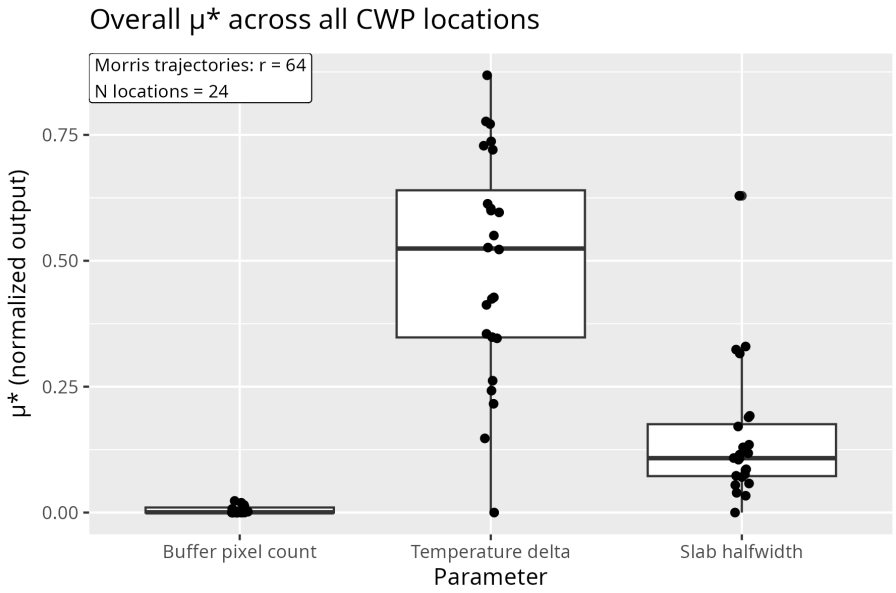

Abb. 2-7: Overall μ^* sensitivity indices for `buffer_px`, `delta_T`, and `step_length` across all classified cold water patches ($N = 24$ locations, $r = 64$ trajectories).

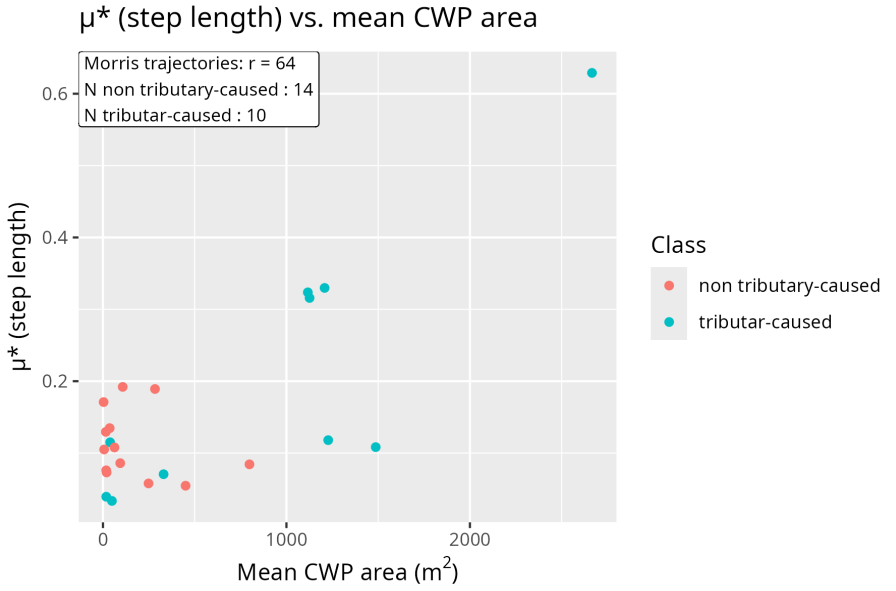

Abb. 2-8: μ^* (step length) versus mean CWP area for each annotated location, colored by patch class.

1.3 Difference in cwp polygons between old and new function

Chapter 2

Optimisation of the CWP algorithm based on the morris sensitivity analysis

2.1 Deploying insight from the morris screening

2.2 Operationalisation of Surrounding Stream temperature

2.3 Implementation Detail

2.4 Reduction of parameters

2.4.1 Usage of KI

Claude Opus 4.6 was used as an aid to ensure proper formulation, spelling and grammatical correctness. Further it was used to derive the LaTeX for Images and simulation protocol table.

2.5 Quellenverzeichnis

E. Dürig. Raster deepdive, 2026. URL https://stds-handin.github.io/Raster-Deepdive/. Coursework submission; not peer-reviewed, cited as supporting empirical observation only.

GDAL/OGR contributors. GTiff – GeoTIFF file format, 2026. URL https://gdal.org/en/stable/drivers/raster/gtiff.html.

R. J. Hijmans, A. Brown, and M. Barbosa. extract: Extract values from a SpatRaster, 2024. URL https://rspatial.github.io/terra/reference/extract.html. `terra` package documentation.

ISciences, LLC. exactextract, 2026. URL https://isciences.github.io/exactextract/index.html.

R. Lovelace, J. Nowosad, and J. Muenchow. *Geocomputation with R*. CRC Press, 2019. ISBN 9780203730058.

T. Mieno. R as GIS for economists: Extraction speed comparison, 2022. URL https://tmieno2.github.io/R-as-GIS-for-Economists/extract-speed.html.

O. Tange. GNU Parallel 20240222. Zenodo, Software, Feb. 2024. URL https://doi.org/10.5281/zenodo.10719803.